

W1 ARCH INTRO

Architecture = high-level structure of components, connectors, constraints, interactions. **Exam:** turn scenario nouns into domain objects, verbs into interactions, qualities into constraints.

Model = abstraction w/ purpose; valid iff it answers stakeholder questions. **Views:** logical(classes), process(runtime), development/packages(components), physical(deploy), scenarios(use cases).

Architecture reasoning: stakeholder → concerns → quality attrs → tactics → views → trade-offs.

Do not draw every noun. Choose boundary, responsibilities, multiplicity, lifecycle ownership, variation points.

Quality attrs: modifiability, performance, safety, security, scalability, availability, interoperability, testability. Map to tactics: RBAC/audit, queue/cache, redundancy, interface adapter, layered decomposition.

Concern	Model cue	Diagram
structure	entities/classes	class
workflow	activities/sequence	activity
lifecycle	states/events	state
integration	msgs/orders	seq/comm
variants	features/profile	feature/meta

Past-paper pattern: large socio-technical system; Q1-Q8 repeatedly ask feature, use case, class, collaboration/communication, activity, state, sequence; 24/25 add metamodel/profile/AI agent.

W2 MBSE / MDE

MBSE uses models as primary artifacts for analysis, design, implementation, testing, maintenance. **MDE/IMDD/IMDSB:** models can drive code/config/test generation.

Model levels: CIM(problem/context), PIM(platform independent), PSM(platform-specific), code/deployment.

Transform: source model M_s + rules R → target model/code M_t ; check traceability $M_s \leftrightarrow M_t$.

Model quality = syntactic correctness + semantic consistency + fit-for-purpose + traceability + maintainability.

Automation: code gen, model analysis, validation, simulation, test generation. **Risk:** false precision; generated code only as good as metamodel/rules.

Artifact	Good for	Trap
PIM	logic portable	ignores platform
PSM	deployment detail	vendor lock-in
DSL	domain fit	narrow reuse
UML	shared notation	over-general

Exam answer skeleton: define boundary → list actors/features → identify domain concepts → choose diagram semantics → add constraints/trade-offs.

W3 REQUIREMENTS

Ad hoc reqs are raw client statements; convert to features, actors, use cases, NFRs, constraints, assumptions.

Feature diagram: root → mandatory/optional/or/alternative groups → cross-tree requires/excludes.

Mandatory = system cannot satisfy core goal without it; optional = value-added/conditional; alternative = exactly one; or-group = one or more.

Use case spec: name, scope, primary actor, stakeholders, preconditions, trigger, main success flow, extensions, exceptions, postconditions, special reqs.

Req quality: complete, consistent, verifiable, feasible, traceable, unambiguous, prioritised.

Common error: writing UI steps only; use case must capture actor goal + system responsibility + alternate/failure flows.

Req phrase	Model element	Check
can configure	use case/feature	actor?
must encrypt	constraint/NFR	verifiable?
supports many devices	scalability	limit?
receives alerts	event flow	source?

Mini example: HMS schedule maintenance: pre auth+device registered; main select device → choose slot → confirm provider; alt no slot; exception unauthorised.

W4 ARCH MODEL

Architecture modelling refines requirements into components, responsibilities, connectors, deployment, interfaces, and quality tactics.

Styles: layered, client-server, microservice, event-driven, pipe-filter, repository, MVC, plugin. Select by forces, not fashion.

Design flow: context diagram → component boundary → interface contracts → data ownership → deployment/security constraints → scenarios.

High marks require rationale: why this decomposition supports modifiability/security/performance, and what it sacrifices.

Style	Strength	Weakness
layered	separation	latency/rigid
event	decoupled	debug/order
microservice	scale teams	ops complexity
repo	shared data	bottleneck
plugin	extension	versioning

Connector semantics: call, message, event, data stream, shared DB, adapter. Label direction, protocol, sync/async, error handling. **Trade-off phrase:** choose event-driven for machine alerts because producers/consumers evolve independently; mitigate ordering by idempotent handlers + event log.

W5 STRUCTURE

Class diagram captures static domain model: classes, attrs, ops, associations, multiplicities, generalisation, aggregation/composition, interfaces.

Composition means strong lifecycle ownership. If part can outlive whole, use aggregation/association.

Multiplicity: 1, 0..1, 0..*, 1..*; put on both ends. Use role names when association meaning unclear.

Scenario mining: nouns → candidate classes; repeated states → enum/state machine; external systems → interfaces/adapters; records/events → history classes.

UML relation	Meaning	Example
inherit	is-a	Vehicle subclasses
compose	part dies w/ whole	Order-Line
aggregate	shared part	Team-Member
realize	implements	Adapter-Iface
assoc	knows/uses	User-Report

Anti-pattern: God class "System" connected to everything; replace by modules/services + domain entities.



W6 BEHAVIOUR

Activity diagram: control/data flow; good for workflow, branching, concurrency, exception paths. Use action, decision/merge, fork/join, swimlanes, object nodes.

State machine: lifecycle of one object; states + events + guards + actions. Good for machine, order, shipment, ticket.

Sequence: time-ordered messages across lifelines; good for API/integration/onboarding. **Communication:** same interactions emphasising links + numbered messages.

Do not mix diagrams: activity shows process; state machine shows one object's legal states; sequence shows message order.

Diagram	Must show	Frequent miss
activity	decisions/forks	no end state
state	events/guards	states as actions
seq	lifelines/order	no returns/errors
comm	numbered msgs	no object links

Fault handling: event → classify severity → branch warning/recoverable/critical → notify actor → update audit/schedule. **Guard syntax:** event [condition] / action, e.g. sensorFault [recoverable] / createTicket.

W7 METAMODEL

Metamodel defines modelling language concepts, relations, constraints; model conforms to metamodel: $M \vdash M.M$. **MOF stack:** M3 meta-metamodel; M2 UML/metamodel; M1 domain model; M0 runtime instances.

Metamodel design: identify variable concepts → abstract syntax classes → relations/multiplicities → well-formedness constraints → example instance.

A metamodel is not an ordinary domain class diagram only; it describes valid models/templates/configuration families.

Need	Metaclass	Constraint
custom workflows	WorkflowTemplate	has start/end
governance models	GovernanceRule	owner exists
IP policies	Policy	scope defined
agent goals	Objective	weight/rank set

UML profile: lightweight extension via stereotype, tagged value, constraint on base metaclass. Example: <<MachineType>> on Class, tag capability/Category, constraint at least one operation.

OCL-ish: context MachineType inv: self.operation->size() >= 1.

W8 AD DESIGN

Advanced design modelling covers patterns, variability, product lines, model analysis, simulation, consistency checking, and evolution.

Patterns: Adapter external systems, Strategy scheduling policy, Observer event notifications, State machine control, Factory creates device adapters, Repository audit/event log.

Consistency checks: feature requires component; class supports use case; activity refers to existing operations; sequence messages map to interfaces; states reachable.

Traceability matrix: Requirement → feature → use case → class/component → test. Missing links reveal design holes.

Change	Design support	Evidence
new device	plugin/adaptor	interface
new policy	strategy/profile	tagged rule
new report	analytics service	data contract
new safety rule	state guard	constraint

Analysis answer: mention feasibility, consequences, verification, and model maintenance.

Mini example: add AI route predictor as component using traffic data; expose prediction API; keep fallback heuristic; monitor bias/performance drift.

W9 AGILE / MID

Agile = iterative delivery, feedback, working software, adaptive planning. In MBSE, models must be living artifacts, not phase-gate paperwork.

Scrum: product backlog, sprint backlog, increment, daily scrum, review, retrospective. **Kanban:** WIP limits, flow, cycle time.

Model in sprint: select story → update req/use case → model design slice → implement/generate → test/validate → refactor model.

Agile + MBSE tension: upfront modeling improves consistency; excessive modeling delays feedback. Use just-enough executable/traceable models.

Artifact	Agile role	Exam point
user story	need	actor+goal
acceptance test	verify	observable
model slice	design	traceable
demo	feedback	validate

Pitfall: calling any iterative process agile; must include feedback, prioritisation, adaptation, customer value.

Risk answer: large safety-critical projects need more architecture governance while still using incremental validation.

W10 PLATFORMS

Modern platforms for software modelling: cloud, containers, microservices, IoT/edge, cyber-physical systems, digital twins, low-code/model tools.

Deployment view: nodes, artifacts, communication paths, trust zones. Include latency, bandwidth, offline mode, security boundary. Integration design: external API → adapter → validation → event log → domain service → analytics.

For IoT/smart systems: separate device telemetry, command/control, diagnostics, audit, and user workflow.

Platform issue	Model element	Mitigation
edge offline	state/cache	sync policy
vendor API	interface	adapter
cloud scale	component	queue/autoscale
privacy	constraint	RBAC/log

Sequence diagrams should show auth, validation, failure response, activation, monitoring; not just happy path.

Example onboarding: VendorPortal → AccessControl → Registry → CapabilityModel → DiagnosticAdapter → Approval → Scheduler.

W11 PROCESS MODELS

Process model defines activities, artifacts, roles, controls. Examples: waterfall, V-model, spiral, RUP, agile, DevOps, model-driven lifecycle.

Waterfall: clear phases; weak change handling. **V-model:** early test mapping; good regulated contexts. **Spiral:** risk-driven iterations. **DevOps:** CI/CD + operations feedback.

Process choice must match uncertainty, safety/regulation, team size, stakeholder access, and integration risk.

Context	Process fit	Reason
stable reqs	waterfall	documentation
high risk	spiral	risk prototypes
changing product	agile	feedback
ops critical	DevOps	monitoring

MBSE lifecycle: requirements model → architecture model → detailed UML/DSL → transformation/code → tests/simulation → deployment → model evolution.

Traceability in process: helps impact analysis: if requirement changes, find affected diagrams, classes, tests, deployment nodes.

Failure modes: state model, no model ownership, tool lock-in, over-modelling, missing verification, poor stakeholder readability.

W12 TOPICS

AI agent design in ELEC5620 context: goals, environment, sensors/data, actions, decision strategy, learning/feedback, oversight, failure modes.

Agent architecture: data sources → perception/feature extraction → knowledge/model → planner/optimizer → action recommender → human approval → audit/monitor.

Competing objectives: use weighted sum for transparent scalar trade-off, Pareto for exploring non-dominated options, hierarchy for hard safety/compliance constraints.

Concern	Failure	Mitigation
bias	unfair recs	audits/diversity
privacy	leakage	access+redaction
drift	bad predictions	monitor/retrain
overtrust	automation bias	human-in-loop

Profile answer: stereotype + base metaclass + tag + constraint. Use domain semantics, not decorative labels.

Final exam strategy: allocate time by diagram marks; first list actors/classes/messages in text, then draw. Every diagram must have labels, multiplicities/guards/messages where relevant.

Marking heuristic: 60% correct concepts, 25% relationships/constraints, 15% clarity/rationale. If uncertain, state assumption explicitly.

DIAGRAM CHECKLIST

Feature: root, groups, mandatory/optional, requires/excludes. **Use case:** pre/main/alt/exception/post. **Class:** multiplicities, ownership, interfaces. **Activity:** decisions/forks/swimlanes. **State:** events/guards/actions. **Seq/Comm:** objects, messages, order.

Meta/Profile: metaclasses, constraints, stereotypes.

One-page memory: feature = what varies; class = what exists; activity = what happens; state = how one thing evolves; sequence/communication = who talks to whom; metamodel/profile = what models are legal.

UML SYNTAX

Class box: name / attributes / operations. Attr: visibility name:type (mult) = default. Op: name(params):return. Visibility: + public, - private, # protected, ~ package.

Association: solid line; navigability arrow optional; role names useful; multiplicity at opposite end answers "how many of that object per this object"

Composition ♦ at owner end; aggregation ◇ at whole end; generalisation hollow triangle toward superclass/interface.

Interface: <<interface>> or lollipop; use when concrete device/vendor implementations vary behind stable contract.

Abstract class: italic name or {abstract}; good for Machine, User, Report, Event. Enum for Status/Severity, not subclass explosion.

Constraint notation: {ordered}, {unique}, guard [batteryLow], invariant {<total>0}.

Symbol	Read as	Use when
1..*	one or more	order lines
0..1	optional	current machine
0..*	many/none	events
1	exactly one	owner

FEATURE ANSWERS

Full-credit recipe: root system → 5-8 top groups → 2-4 subfeatures each → mark mandatory/optional/alt/or → add 2-4 cross-tree deps → explain assumptions.

Typical groups: User/RBAC, Order/Workflow, Resource/Device; Scheduling/Optimisation; Monitoring/Analytics; Maintenance/Quality; Integration; Security/Audit.

Dependency examples: PredictiveMaintenance requires SensorData; VendorDiagnostics requires VendorAccess; Analytics requires DataCollection; VoiceControl excludes NoAudioInterface.

Feature diagrams are not class diagrams: avoid listing every entity; show user-visible capabilities and variability.

Marking words: complete hierarchy, meaningful grouping, correct optionality, useful constraints, no impossible combinations.

USE CASE TEMPLATE

Name: verb phrase tied to actor goal. **Primary actor:** stakeholder initiating value. **Scope:** system boundary. **Level:** user-goal.

Main flow: trigger → authenticate/select → system validates → actor provides info → system commits/update/notify → postcondition.

Alt flow: valid but different path, e.g. no slots, insufficient materials, low battery. **Exception:** failure path, e.g. unauthorized, unavailable external service, safety stop.

Preconditions must be true before start; postconditions true after success/failure. Do not repeat main-flow steps as preconditions.

Section	Good answer	Bad answer
actor	role name	"user" only
pre	released order	future result
alt	recoverable path	same as main
post	audit updated	vague success

CLASS PATTERNS

Order domain: Customer/Importer/Planner, Order, OrderLine/WorkOrder, Item/Batch, Payment/Invoice, Shipment, Warehouse, Supplier.

Smart systems: Device/Machine abstract, SensorReading, Event, Fault, MaintenanceTicket, Capability, Adapter, DiagnosticReport.

Analytics: DataSource, Metric, Dashboard, Report, PredictionModel, Recommendation, AuditLog.

Relationship heuristics: contains/lifecycle → composition; is-a taxonomy → inheritance; can play many roles → Role class/RBAC not subclasses; external API → interface+adapter.

Add at least one interface in ELEC5620 class diagrams: DiagnosticInterface, PaymentGateway, TrafficDataProvider, NotificationService.

Multiplicity sanity: Order has 1..* line items; User has 0..* sessions; Machine has 0..* events; WorkOrder has 0..1 current machine.

INTERACTION TEMPLATES

Sequence diagram lifelines: Actor/UI, Auth, Controller/Service, Registry/Repository, ExternalSystem, Analytics/Notifier.

Happy path: request → validate/auth → retrieve model/data → decide/compute → update state → notify/log → response.

Add fragments: alt for success/failure; loop for repeated items; par for concurrent notifications; opt. for optional analytics.

Communication diagram: objects as nodes; links + numbered messages: 1, 1.1, 1.2, 2. Use verbs on messages.

Collaboration vs communication in papers: if asked collaboration+object diagram, show roles/classes then concrete objects linked by those roles.

Message names: submit(), validate(), reserve(), allocate(), notify(), createTicket(), updateStatus(), generateReport(), approve(), activate().

ACTIVITY / STATE

Activity: start/end, action nodes, decision/merge diamonds, fork/join bars, swimlanes. Use object nodes for Order, Ticket, Batch where helpful.

Activity scoring: cover normal path + at least two decisions + exception/escalation + final outcome.

Reusable flows: search/select/order; detect/classify/escalate; schedule/execute/complete; register/validate/approve/activate.

State machine: choose one entity only. States should be stable conditions, not tiny actions. Transitions labelled event [guard] / action.

Object	States	Events
Machine	ordered/inTransit/delivered	ship/clear
Ticket	idle/setup/running/fault	start/fault
Batch	open/assigned/resolved	assign/fix
	pending/hold/rework/scrap	inspect

If a diagram starts to describe multiple objects' lifecycles, split into sequence/activity or pick the central object.

META / PROFILE

Metamodel answer: define modelling concepts, not implementation classes. For configurable systems, use UML for configuration, DataSharingPolicy, IPAgreement. **Metamodel steps:** explain purpose, multiplicities → constraints → **Constraint examples:** each WIP milestone; each DataSharingPolicy Objective has priority; each Agent has role. **UML profile answer:** <<Sensor>> tagged values + constraints.



Tagged	Base	Tag/constraint
MachineType	Class	capability: ops=0
SafetyEvent	Signal	severity: stop transition
DiagnosticAdapter	Component	protocol: approved
InspectionRule	Constraint	tolerance: linked plan

Profile pitfall: <<Order>> on class with no tag/constraint adds no modelling power.

AI AGENT DESIGN

PEAS-like: Performance objectives; Environment; Actuators/actions; Sensors/data. State assumptions about observability, uncertainty, and human authority.

Agent workflow: collect data → clean/feature → predict/evaluate → optimise alternatives → explain recommendation → human approves → monitor outcome.

Decision strategies: weighted sum = transparent and simple; Pareto = exposes trade-off frontier; hierarchy = hard constraints first; rule+ML hybrid = safety guardrails.

For safety/ethics, put compliance/privacy/safety as hard constraints, then optimise efficiency/impact.

Objective conflict	Bad choice	Better
impact vs diversity	single metric	Pareto/weights
budget vs safety	cheap only	hard safety
speed vs privacy	raw sharing	redaction/RBAC
automation vs trust	auto-final	human review

Failure modes: biased data, privacy leakage, feedback loops, model drift, adversarial gaming, unclear accountability, automation bias.

SOURCE TRENDS

2021 HMS: smart home, stakeholders, maintenance, firmware, service provider, analytics, security. **2023 IFMS:** fleet, route optimisation, charging, predictive maintenance, traffic integration.

2024 LSCM: supply chain, orders, warehouse, transportation, partnership analytics, plus IRCP metamodel + AI agent. **2025 SMS:** manufacturing orders, machines, quality, faults, vendor onboarding, UML profile.

Trend: scenario changes, diagram family stable. Newer papers reward metamodel/profile and agent trade-off reasoning.

Preparation: memorise templates; practise translating one new scenario into 8 diagrams in 120 min; write assumptions instead of freezing.

MINI EXAMPLES

Feature dep: RouteOptimisation requires RealTimeTrafficData; PredictiveMaintenance requires VehicleTelemetry.

Use-case alt: if charging station unavailable, system proposes nearest available slot and logs schedule conflict.

Class relation: FleetManager inherits User? Better: User has Role; FleetManager is role if same person can be Driver+Manager.

Activity branch: [measurement within tolerance] accept batch; [minor deviation] quality review; [critical defect] hold+scrap/rework.

State transition: Running → sensorFault(recoverable)/pause+notify → Fault; Fault → resetOK/clearTicket → Running.

Sequence alt: CityTrafficSystem reports closure; IFMS validates event; RouteService recomputes; DriverApp receives updated route; Analytics logs impact.

Last-minute rule: every answer should say who, what object, what event/message, what constraint, and what final state.